Unit 3 - Calculus
3.2 Optimization and Linear Regression

Graham West

MSDS 520: Math and Stats Foundations for Data Science
Meharry Medical College

Outline

1. Why Optimize?
2. The Language of Optimization
3. The Derivative Test
4. The Second Derivative Test
5. Linear Regression
6. Solving Linear Regression: the Normal Equations
7. Gradient Descent
8. Gradient Descent in Two Variables
9. Solving Linear Regression: Gradient Descent
10. Review

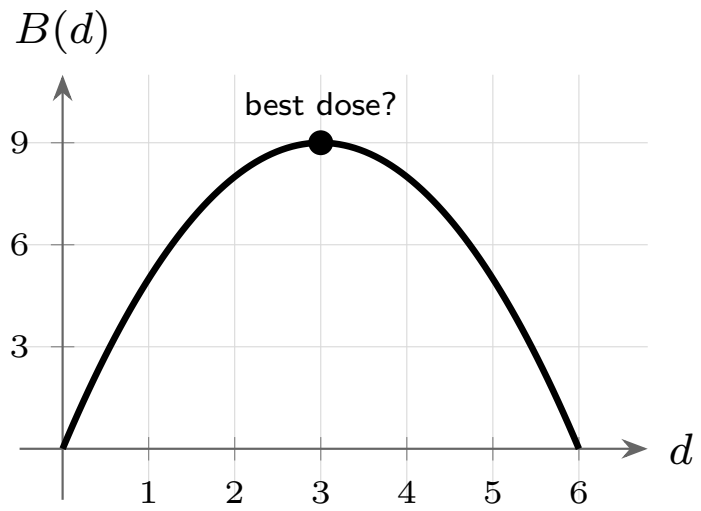
Why Optimize?

A Problem to Motivate Us

Choosing a dose

- More drug gives **more benefit**.
- More drug also gives **more side effects**, and those grow faster.
- A simplified model of the **net benefit** of a dose of d mg:

$$B(d) = 6d - d^2$$



Two questions this lecture answers

1. Where is the peak, **exactly**, without plotting? $\Rightarrow$ the **derivative test**
2. What if the formula is too complicated to solve? $\Rightarrow$ **gradient descent**

Where This Shows Up in Data Science

Optimization underneath the tools

- **Training a model.** A machine-learning model is **trained** by making its **loss** (how wrong its predictions are) as small as possible.
- **Fitting a line to data.** The best-fit line is the one with the smallest squared error. This lecture ends by building it.
- **Treatment planning.** Radiation therapy: maximize the dose to a tumor, minimize the dose to healthy tissue.
- **Hospital operations.** Staff schedules, bed assignments, supply orders.
- **Finance and engineering.** Portfolios (most return, least risk), aircraft wings (least drag).

Min or max: the same problem

Making f as **large** as possible is making $-f$ as **small** as possible. Machine learning always phrases it as a **minimum**.

What We Will Build

The plan

1. The **language**: objective function, search space, minimizer, local vs. global.
2. The **derivative test**: peaks and valleys are where $f'(x) = 0$.
3. The **second derivative test**: curvature (Unit 3-1) sorts the candidates.
4. **Linear regression**: the model, its residuals, and the loss $L(m, b)$.
5. **Solving it with the normal equations**: set the partials of the loss to zero.
6. **Gradient descent**: when $f'(x) = 0$ cannot be solved, walk downhill instead.
7. The **learning rate**, when to stop, and the local-minimum trap.
8. **Gradient descent in two variables**: step against the gradient (Unit 3-1).
9. **Solving it with gradient descent**: the same line, and why it matters.

Notebooks for this lecture: `U3-5_Optimize-1_DerivativeTest`, `U3-7_Regression-1` to `-6`, `U3-7_GradientDescent-1_Descent`.

The Language of Optimization

Optimization Problems

The problem

Find the smallest or largest value of a function $f(x)$ over a set of allowed inputs D. Each answer is a **number**, the output at some best input x^* in D:

$$\min_{x \in D} f(x) = f(x^*), \qquad \text{where } f(x^*) \leq f(x) \text{ for every } x \text{ in } D$$

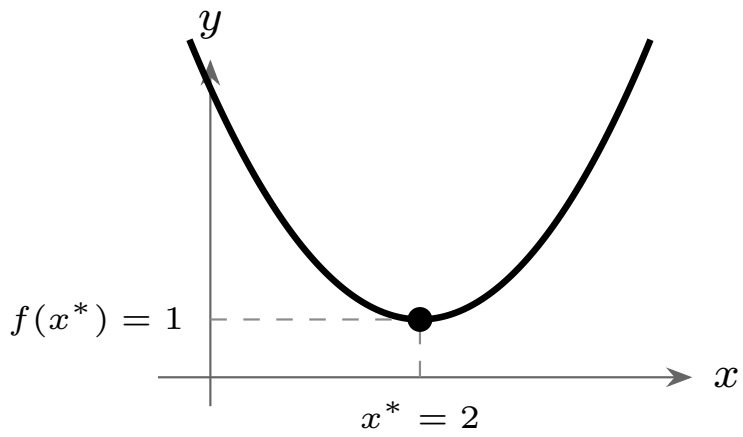
$$\max_{x \in D} f(x) = f(x^*), \qquad \text{where } f(x^*) \geq f(x) \text{ for every } x \text{ in } D$$

- f is the **objective function**: the score we care about.
- D is the **search space**: the inputs we are allowed to try.
- A minimum or a maximum is called an **extremum**.
- We usually **minimize**. Any max problem becomes a min problem by flipping the sign: maximizing f is minimizing $-f$, with the same x^*.

Two Answers: Where, and How Good

A minimization has two answers

- The **minimizer** x^*: **where** the minimum happens.
- The **minimum value** $f(x^*)$: **how low** it gets.
- For $f(x) = (x-2)^2 + 1$: $x^* = 2$ and $f(x^*) = 1$.



Local vs. Global

Global minimum

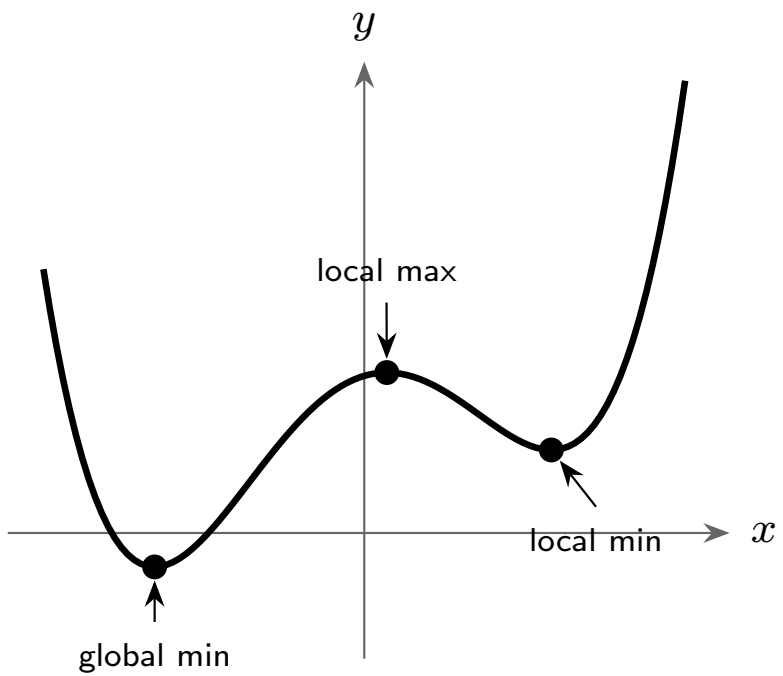
$f(x^*) \leq f(x)$ for **every** x in D: the lowest point anywhere.

Local minimum

$f(x^*) \leq f(x)$ for every x **near** x^*.

- You can't get better without first getting worse.
- Every global minimum is also local, not the other way around.

Local and global **maxima** are defined the same way, with $\geq$.



Why the Difference Matters

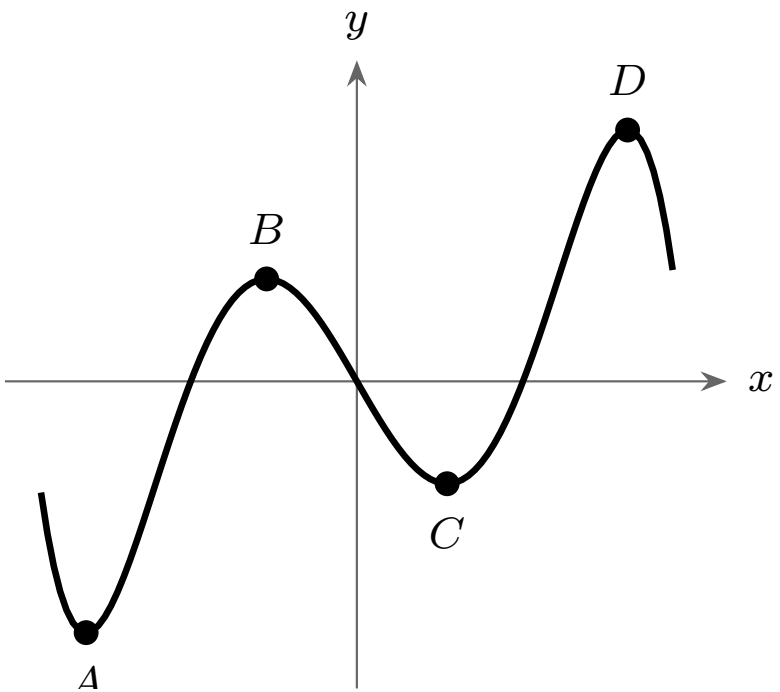
An algorithm walks in fog

- It sees only the ground **right under its feet**: the value and slope where it stands.
- At a local minimum, **every** small step goes uphill, so it looks exactly like the bottom.
- An algorithm can stop at a local minimum and report it as **the** answer.

Why not just plot it and look?

- With **one** input, a plot settles it.
- A machine-learning model has thousands to billions of inputs. There is no picture to look at.
- Most of the time we **cannot confirm** we found the global minimum. In practice the goal is a minimum that is **low enough**.

Exercise: Read the Graph



The search space is the window shown, $-3.5 \leq x \leq 3.5$.

Try it

1. Classify each of A, B, C, D as a local minimum or a local maximum.
2. Which point is the **global** minimum on this window? The global maximum?
3. Which points are local but **not** global? Why?
4. What do the tangent lines at all four points have in common?

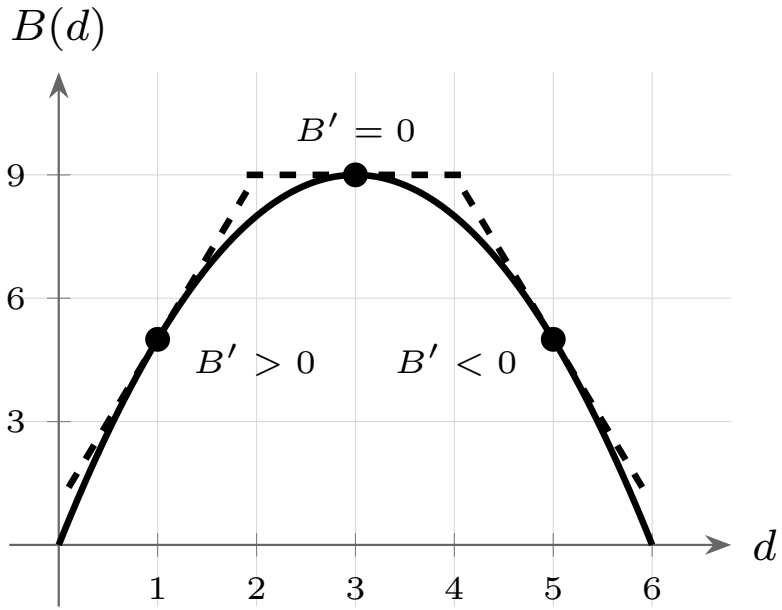
Exercise: Solution

Answers

1. Look at the neighbors of each point:
   - A and C sit at the bottom of a valley: every nearby point is higher. **Local minima.**
   - B and D sit at the top of a hill: every nearby point is lower. **Local maxima.**
2. A is the lowest point anywhere on the window: the **global minimum**. D is the highest: the **global maximum**.
3. B and C. B is the top of its own hill, but D is higher. C is the bottom of its own valley, but A is lower. Each is best only **nearby**.
4. All four tangent lines are **horizontal**: slope 0, so $f'(x) = 0$ at every one of them. That is the key to finding extrema with calculus, and it is the next section.

The Derivative Test

At a Peak, the Tangent Is Flat



Dashed: tangent lines at $d = 1, 3, 5$.

Follow the slope over the hill

- Left of the peak: rising, $B'(d) > 0$.
- At the peak: **flat**, $B'(d) = 0$.
- Right of the peak: falling, $B'(d) < 0$.

To turn from rising to falling, the slope has to pass through 0.

Critical point

A point c where $f'(c) = 0$. At a smooth max or min **inside** D, the point is always a critical point.

The Derivative Test

Algorithm: find the extrema of f on D

1. **Differentiate**: find $f'(x)$.
2. **Solve** $f'(x) = 0$. The solutions are the **critical points**: the candidates.
3. If D has **endpoints**, add them to the candidates.
4. **Evaluate** f at every candidate. The largest value is the max, the smallest is the min.

Why this works

- A smooth function can only turn around where its slope is 0.
- So instead of trying **infinitely** many x values, we check a **short list**.

Notebook `U3-5_Optimize-1_DerivativeTest`, §1 and §2

Worked: The Best Dose

The steps, for $B(d) = 6d - d^2$

1. Differentiate: $B'(d) = 6 - 2d$.
2. Solve: $6 - 2d = 0$, so $2d = 6$, so $d = 3$.
3. No endpoints yet: any dose is allowed.
4. Evaluate: $B(3) = 18 - 9 = 9$.

Is it a max?

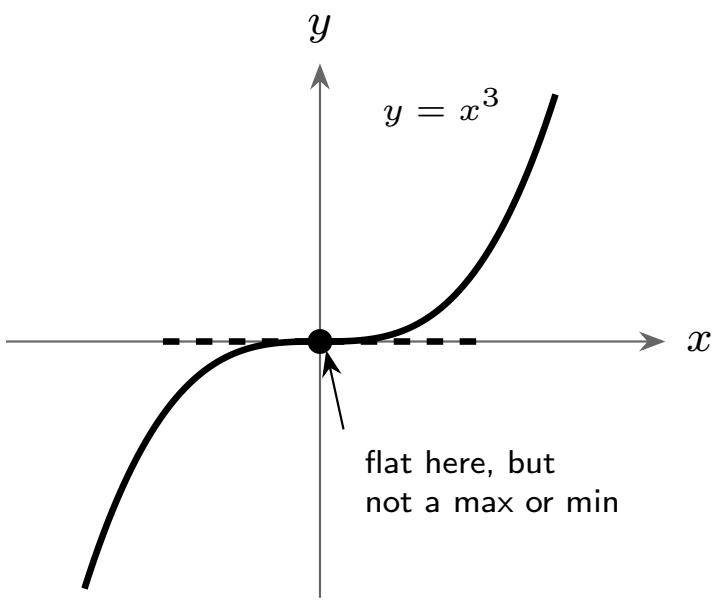
d	$B(d)$	$B'(d)$
2	8	$+2$
3	$\mathbf{9}$	0
4	8	-2

Lower on both sides: a **maximum**.

What it means

- Below 3 mg, more drug helps: $B'(d) > 0$.
- Above 3 mg, the side effects win: $B'(d) < 0$.
- 3 mg is the best tradeoff, with net benefit 9. We found it **without** plotting.

Flat Does Not Mean Extreme



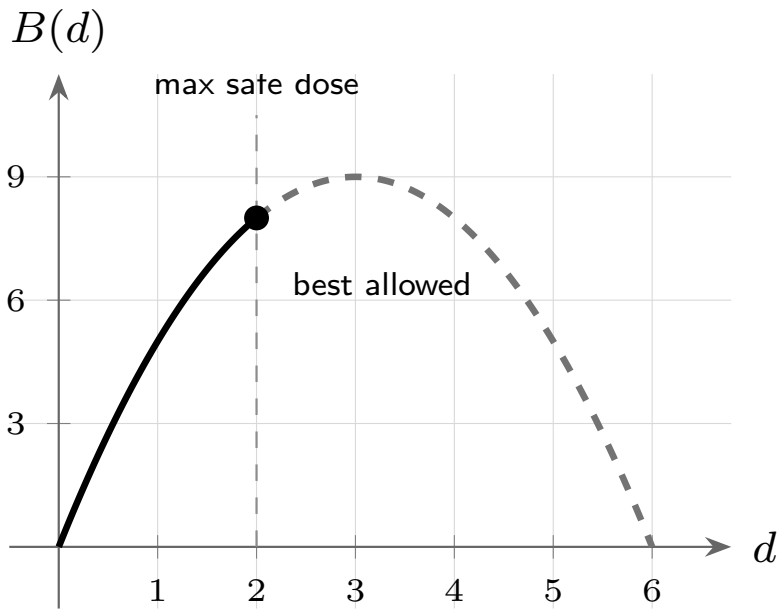
The example $f(x) = x^3$

- $f'(x) = 3x^2$, and $3x^2 = 0$ gives $x = 0$: a critical point.
- But $f(-0.1) < f(0) < f(0.1)$: the curve **rises on both sides**.
- It pauses flat, then keeps climbing. No turn, so no extremum.

Watch out

$f'(x) = 0$ gives **candidates**, not answers. Always check each one.

Watch the Endpoints



Solid: allowed doses. Dashed: not allowed.

Now the dose is capped at 2 mg

The search space is $D = [0, 2]$.

- $B'(d) = 6 - 2d > 0$ for every d in D: still rising.
- The critical point $d = 3$ is **outside** D.
- Candidates are the endpoints: $B(0) = 0$, $B(2) = 8$.
- Best: $d = 2$, where $B'(2) = 2 \neq 0$.

Watch out

With a restricted D (**constrained optimization**), the best value can sit on the **boundary**, where $f' \neq 0$.

Exercise: The Derivative Test

Try it

1. Find the critical point of $f(x) = x^2 - 8x + 20$. Is it a max or a min? What is the value there?
2. A clinic's daily profit, in hundreds of dollars, when it books q appointment slots is

$$P(q) = -2q^2 + 40q - 50.$$

   How many slots maximize profit, and what is the profit?
3. The clinic only has staff for **at most 5** slots, so $D = [0, 5]$. Now what is the best q?

Exercise: Solution

Answers

1. $f'(x) = 2x - 8 = 0$, so $x = 4$, and $f(4) = 16 - 32 + 20 = 4$. Neighbors: $f(3) = 5$ and $f(5) = 5$, both higher, so it is a **minimum**: the lowest value of f is 4, at $x = 4$.

2. $P'(q) = -4q + 40 = 0$, so $q = 10$, and $P(10) = -200 + 400 - 50 = 150$. Neighbors: $P(9) = P(11) = 148$, both lower, so it is a **maximum**. Book 10 slots for a profit of $\$15{,}000$ a day.

3. $P'(q) = -4q + 40 > 0$ for every q in $[0, 5]$: profit is **still rising** at the cap, and the critical point $q = 10$ is outside D. Check the endpoints:

   - $P(0) = -50$ and $P(5) = -50 + 200 - 50 = 100$.
   - Best: $q = 5$, profit $\$10{,}000$, at the **boundary**, where $P'(5) = 20 \neq 0$.

   Every extra slot up to the cap still adds profit; the staffing limit, not the market, is what stops us.

The Second Derivative Test

The Second Derivative Test

At a critical point c, where $f'(c) = 0$:

- $f''(c) > 0$: convex, a cup. — c is a **local min**.
- $f''(c) < 0$: concave, a cap. — c is a **local max**.
- $f''(c) = 0$: the test **says nothing**. — Check by hand.

Watch out: all three have $f'(0) = f''(0) = 0$, yet at 0 they differ

$f(x) = x^3$	$f(x) = x^4$	$f(x) = -x^4$
neither	min	max

The whole curve: local becomes global

- **Convex** everywhere ($f'' \geq 0$): a critical point is a **global min**.
- **Concave** everywhere ($f'' \leq 0$): a critical point is a **global max**.
- The best dose: $B''(d) = -2$ for every d, so $d = 3$ is the **global max**.

Notebook `U3-5_Optimize-1_DerivativeTest`, §3

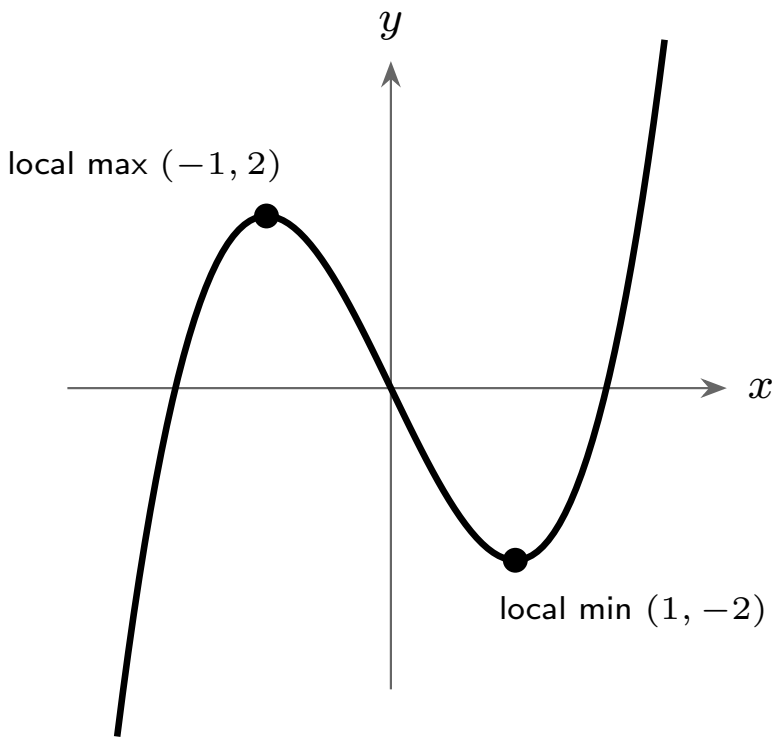
Worked: $f(x) = x^3 - 3x$

The steps

1. $f'(x) = 3x^2 - 3 = 3(x^2 - 1)$.
2. $3(x^2 - 1) = 0$ gives $x = -1$ and $x = 1$.
3. $f''(x) = 6x$:
   - $f''(-1) = -6 < 0$: a **local max**, $f(-1) = -1 + 3 = 2$.
   - $f''(1) = 6 > 0$: a **local min**, $f(1) = 1 - 3 = -2$.

Local, not global

Past $x = 2$ the curve climbs above 2 for good ($f(3) = 18$), so this max is only **local**.



Exercise: The Second Derivative Test

Try it

1. Find the critical points of $f(x) = 2x^3 - 6x^2 + 1$ and classify each with the second derivative test.
2. Show that the test says nothing for $f(x) = x^4$ at $x = 0$. Is $x = 0$ a min, a max, or neither?
3. A patient's temperature $T(t)$, t hours after admission, has $T'(6) = 0$ and $T''(6) < 0$. In plain words, what happened at hour 6?

Exercise: Solution

Answers

1. $f'(x) = 6x^2 - 12x = 6x(x-2)$, which is 0 at $x = 0$ and $x = 2$. Then $f''(x) = 12x - 12$:
   - $f''(0) = -12 < 0$: a **local max**, with $f(0) = 1$.
   - $f''(2) = 24 - 12 = 12 > 0$: a **local min**, with $f(2) = 16 - 24 + 1 = -7$.
2. $f'(x) = 4x^3$ and $f''(x) = 12x^2$, so $f'(0) = 0$ and $f''(0) = 0$: the test says nothing. By hand: $x^4 \geq 0 = f(0)$ for every x, so $x = 0$ is a **min**, in fact the global min.
3. $T'(6) = 0$: the temperature stopped changing for an instant. $T''(6) < 0$: the curve is concave there. So the temperature **peaked** at hour 6: the fever was rising before and falling after.

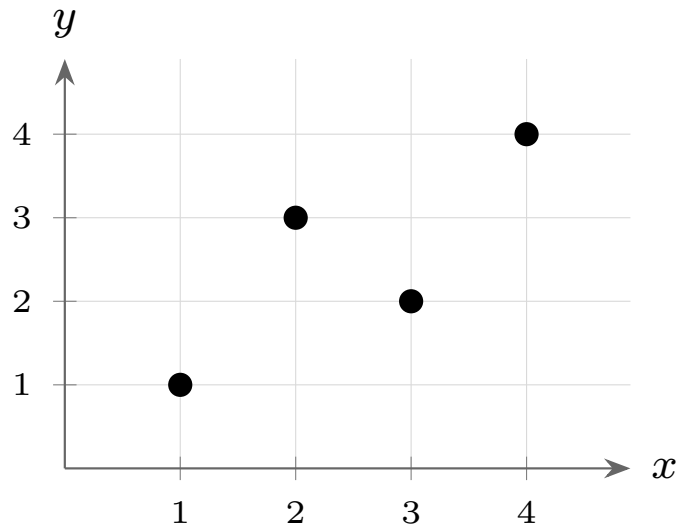
Linear Regression

Four Patients: Which Line Fits Best?

Four patients

patient	dose x (mg)	relief y
1	1	1
2	2	3
3	3	2
4	4	4

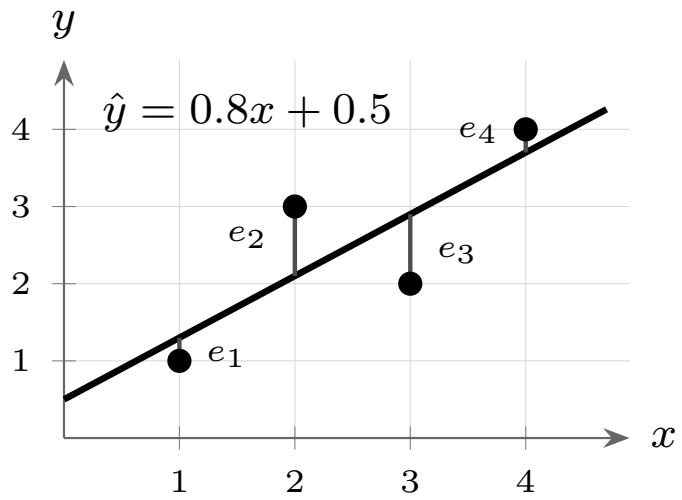
Relief is a symptom score: higher is better.



The question

- The trend is upward, but not a perfect line. **Which** line $\hat{y} = mx + b$ fits best?
- "Best" will be one number, the **loss**, that depends on **two** choices: the slope m and the intercept b.
- Minimizing a function of two variables: **partial derivatives** and the **gradient** (Unit 3-1).

The Model and Its Residuals



x_i	1	2	3	4
y_i	1	3	2	4
$\hat{y}_i$	1.3	2.1	2.9	3.7
$\hat{y}_i - y_i$	0.3	−0.9	0.9	−0.3

The model

$$\hat{y} = mx + b$$

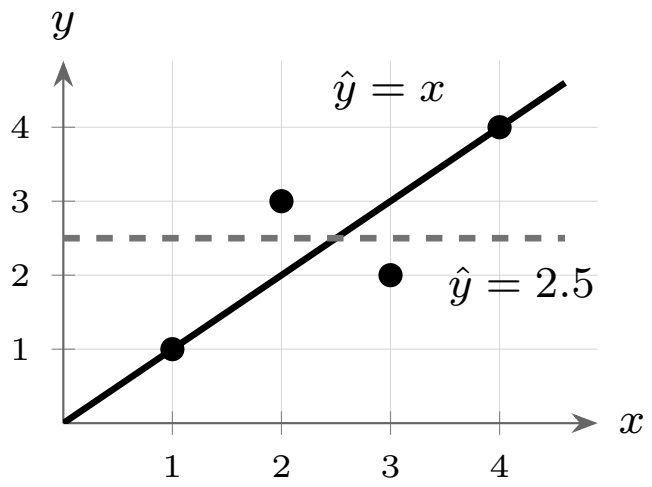
- m is the slope, b the intercept.
- The hat means **predicted**. It is not the unit-vector hat of Unit 2.
- The notebooks write the same line as $\hat{y} = b_0 + b_1 x$.

Residuals: $e_i = \hat{y}_i - y_i$

Predicted minus actual: the vertical gap. Positive for a point below the line, negative above. The table lists them for this candidate line.

Notebook `U3-7_Regression-1_FittingALine`, §2 and §3

Why Square the Residuals?



Solid: $\hat{y} = x$. Dashed: $\hat{y} = 2.5$.

Adding raw residuals fails

Positive and negative misses **cancel**: both lines total 0, a "perfect" score, yet the flat line plainly fits worse.

Squaring fixes it

Every miss counts as positive, and **big** misses cost more: a miss of 2 costs 4, a miss of 0.5 only 0.25.

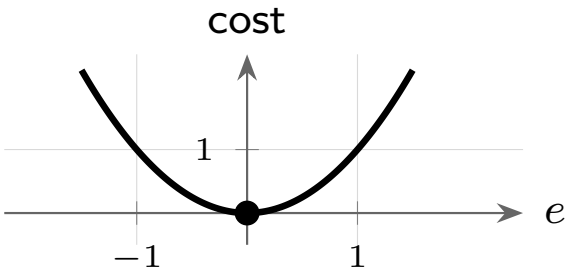
Two lines, same data

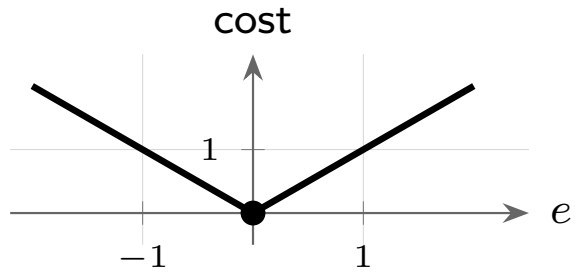
line	**residuals** $\hat{y}_i - y_i$	$\sum(\hat{y}_i - y_i)$	$\sum(\hat{y}_i - y_i)^2$
$\hat{y} = x$	$0,\ -1,\ 1,\ 0$	0	2
$\hat{y} = 2.5$	$1.5,\ -0.5,\ 0.5,\ -1.5$	0	5

Notebook `U3-7_Regression-1_FittingALine`, §3

Why Squares, Not Absolute Values

A residual $e = \hat{y} - y$ can be negative. Two ways to make every miss a positive cost:





Cost e^2

- Derivative $2e$ exists **everywhere**.
- Flat bottom: $2e = 0$ finds it.
- The slope shrinks near the bottom, signaling how close you are.

Cost $|e|$

- Derivative ± 1, **none** at $e = 0$ (Unit 3-1).
- Sharp bottom: nothing to solve.
- The slope never shrinks: no signal of how close you are.

Why we care

Fitting a line means finding the bottom of its loss, so we square the misses. Absolute error is still used, but it needs other tools.

The Loss Function

Half the sum of squared residuals

$$L(m,b) = \frac{1}{2}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2 = \frac{1}{2}\sum_{i=1}^{n}(mx_i + b - y_i)^2$$

Each term is a squared residual e_i^2. Scaling by $\frac{1}{2}$, or by $\frac{1}{n}$ (**mean squared error**), keeps the same lowest point.

Our four patients, one term each

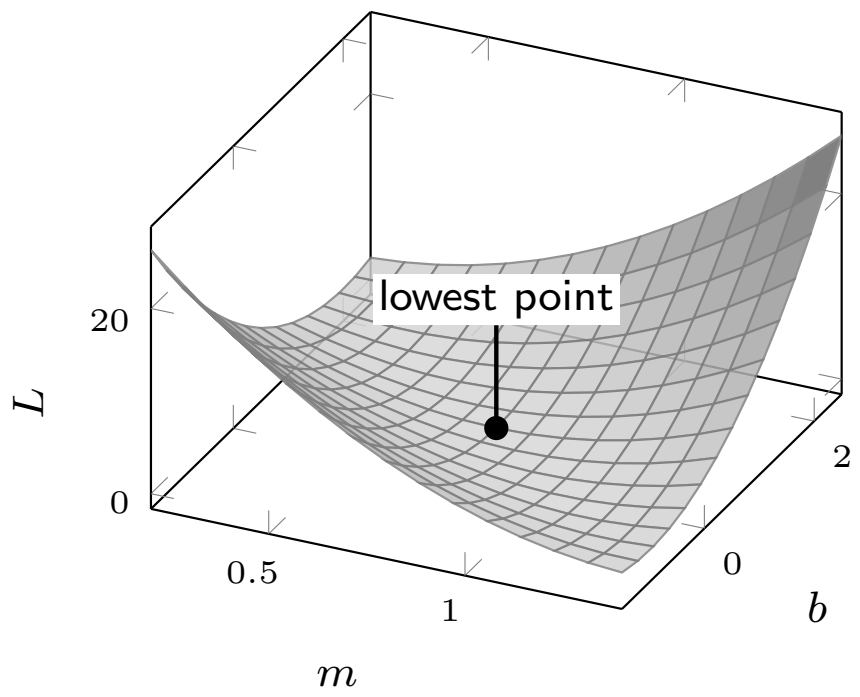
$$L(m,b) = \frac{1}{2}\begin{bmatrix} (\ m + b - 1)^2 \\ +(2m + b - 3)^2 \\ +(3m + b - 2)^2 \\ +(4m + b - 4)^2 \end{bmatrix}$$

Scoring three lines

line	L
$\hat{y} = 2.5$	2.5
$\hat{y} = x$	1
$\hat{y} = 0.8x + 0.5$	0.9

The key shift: the data x_i, y_i are **fixed numbers**; we choose only m and b. So L is a **function of two variables**, and the best line is its **lowest point**.

The Loss Is a Bowl



What the surface says

- **One** lowest point: that pair (m, b) is the best line. We solve for it two ways: the normal equations, then gradient descent.
- **Convex** (Unit 3-1), so no local minima to get stuck in: for a line with squared error, the loss is always a bowl with one bottom.
- A long, narrow **valley**: raise m and lower b together and the line pivots near the middle of the data, so the loss barely changes.

Notebook `U3-7_Regression-3_ThreeWays`, §1

Solving Linear Regression: the Normal Equations

The Partial of the Loss with Respect to m

Step by step: the data x_i, y_i are fixed numbers, and b is frozen

$$\frac{\partial L}{\partial m} = \frac{\partial}{\partial m}\frac{1}{2}\sum_{i=1}^{n}(mx_i + b - y_i)^2$$ start from the loss

$$= \frac{1}{2}\sum_{i=1}^{n}\frac{\partial}{\partial m}(mx_i + b - y_i)^2$$ constants ride along, sums split

$$= \frac{1}{2}\sum_{i=1}^{n}2(mx_i + b - y_i)\cdot x_i$$ chain rule: inner derivative x_i

$$= \sum_{i=1}^{n}x_i\,(mx_i + b - y_i)$$ the $\frac{1}{2}$ cancels the 2

$$= \sum_{i=1}^{n}(mx_i^2 + bx_i - x_iy_i)$$ multiply out

$$= m\sum_{i=1}^{n}x_i^2 + b\sum_{i=1}^{n}x_i - \sum_{i=1}^{n}x_iy_i$$ separate sums; m, b pull out

The Partial of the Loss with Respect to b

Same steps, now with m frozen: only the inner derivative changes

$$\frac{\partial L}{\partial b} = \frac{\partial}{\partial b}\frac{1}{2}\sum_{i=1}^{n}(mx_i + b - y_i)^2 \qquad \text{start from the loss}$$

$$= \frac{1}{2}\sum_{i=1}^{n} 2(mx_i + b - y_i)\cdot 1 \qquad \text{chain rule: inner derivative } 1$$

$$= \sum_{i=1}^{n}(mx_i + b - y_i) \qquad \text{the } \tfrac{1}{2} \text{ cancels the } 2$$

$$= \sum_{i=1}^{n} mx_i + \sum_{i=1}^{n} b - \sum_{i=1}^{n} y_i \qquad \text{separate sums}$$

$$= m\sum_{i=1}^{n} x_i + nb - \sum_{i=1}^{n} y_i \qquad m \text{ pulls out; } b \text{ added } n \text{ times is } nb$$

The Normal Equations

What the two partials say

$mx_i + b - y_i$ is $\hat{y}_i - y_i$, so each partial is a total of the misses:

$$\frac{\partial L}{\partial m} = \sum_{i=1}^{n} x_i \, (\hat{y}_i - y_i), \qquad \frac{\partial L}{\partial b} = \sum_{i=1}^{n} (\hat{y}_i - y_i)$$

Set both to zero: the normal equations

At the bottom of the bowl both slopes are zero. Set each split form to 0, and move the y term across:

$$\begin{aligned} \left(\sum x_i^2\right) \boldsymbol{m} \;+\; \left(\sum x_i\right) \boldsymbol{b} &= \sum x_i y_i \\ \left(\sum x_i\right) \boldsymbol{m} \;+\; n\, \boldsymbol{b} &= \sum y_i \end{aligned}$$

Each parenthesis is just a number from the data, so this is a **pair of linear equations** in $\boldsymbol{m}$ and $\boldsymbol{b}$ (Unit 2-1). Squaring is what made the partials linear.

Matrix Form: Stack the Data

One row per patient

$$X = \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix}, \quad \bar{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}, \qquad X \begin{bmatrix} m \\ b \end{bmatrix} = \bar{y} \text{ means } \begin{bmatrix} mx_1 + b \\ \vdots \\ mx_n + b \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

With n equations and only 2 unknowns, there is usually **no exact solution**.

Multiply both sides by $X^\top$, row dotted with column: the sums appear

$$X^\top X = \begin{bmatrix} x_1 & \cdots & x_n \\ 1 & \cdots & 1 \end{bmatrix} \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} = \begin{bmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & n \end{bmatrix}$$

$$X^\top \bar{y} = \begin{bmatrix} x_1 & \cdots & x_n \\ 1 & \cdots & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} \sum x_i y_i \\ \sum y_i \end{bmatrix}$$

Matrix Form: What Unit 2-5 Promised

The normal equations are one matrix equation

$$\begin{bmatrix} \sum x_i^2 & \sum x_i \\ \sum x_i & n \end{bmatrix} \begin{bmatrix} \boldsymbol{m} \\ \boldsymbol{b} \end{bmatrix} = \begin{bmatrix} \sum x_i y_i \\ \sum y_i \end{bmatrix} \quad \text{is} \quad X^\top X \begin{bmatrix} \boldsymbol{m} \\ \boldsymbol{b} \end{bmatrix} = X^\top \bar{y}$$

When $X^\top X$ has an inverse, the best line is

$$\begin{bmatrix} m \\ b \end{bmatrix} = (X^\top X)^{-1} X^\top \bar{y}$$

More features just add columns to X.

The notebook puts the ones column first, since it writes b_0 first.

Notebook `U3-7_Regression-4_MatrixForm`, §1 to §3

Solving the Running Example

The sums

x_i	y_i	x_i^2	$x_i y_i$
1	1	1	1
2	3	4	6
3	2	9	6
4	4	16	16
10	**10**	**30**	**29**

and $n = 4$.

For our four patients

$$X = \begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \\ 4 & 1 \end{bmatrix}, \qquad \bar{y} = \begin{bmatrix} 1 \\ 3 \\ 2 \\ 4 \end{bmatrix}$$

$$X^\top X = \begin{bmatrix} 30 & 10 \\ 10 & 4 \end{bmatrix}, \quad X^\top \bar{y} = \begin{bmatrix} 29 \\ 10 \end{bmatrix}$$

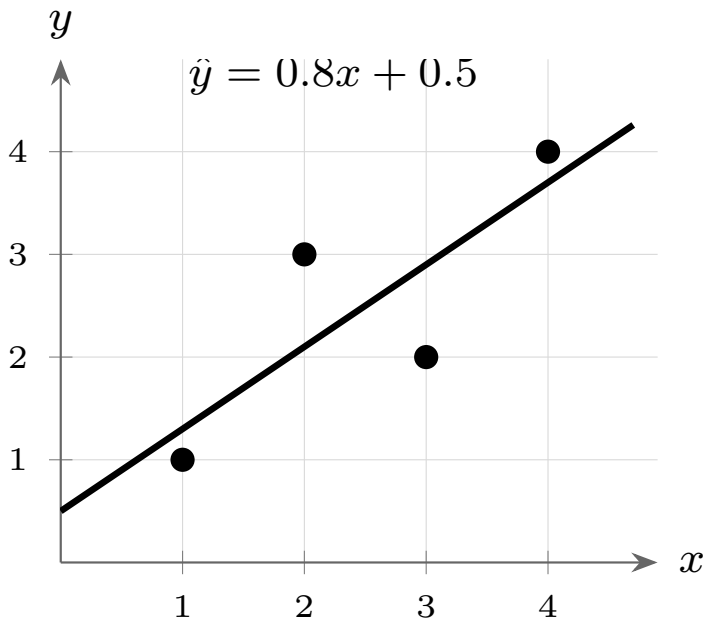
Solve with the Unit 2-5 inverse

$\det(X^\top X) = 30 \cdot 4 - 10 \cdot 10 = 20 \neq 0$, so there is **exactly one** best line:

$$\begin{bmatrix} m \\ b \end{bmatrix} = \frac{1}{20} \begin{bmatrix} 4 & -10 \\ -10 & 30 \end{bmatrix} \begin{bmatrix} 29 \\ 10 \end{bmatrix} = \frac{1}{20} \begin{bmatrix} 16 \\ 10 \end{bmatrix} = \begin{bmatrix} 0.8 \\ 0.5 \end{bmatrix}$$

Notebook `U3-7_Regression-3_ThreeWays`, §3

What the Best Line Says



Reading m and b

- The best line is $\hat{y} = 0.8x + 0.5$: the candidate we drew earlier.
- **Slope** $m = 0.8$: each extra mg of dose raises predicted relief by 0.8 points.
- **Intercept** $b = 0.5$: predicted relief at dose 0. No patient had dose 0, so read it with care.
- **Prediction**: a 2.5 mg dose gives $0.8(2.5) + 0.5 = 2.5$.

Checks

- Its loss, 0.9, beats both earlier lines (1 and 2.5), and every other line.
- Its residuals $\hat{y}_i - y_i$, which are 0.3, -0.9, 0.9, -0.3, sum to 0: exactly what $\partial L/\partial b = 0$ says.

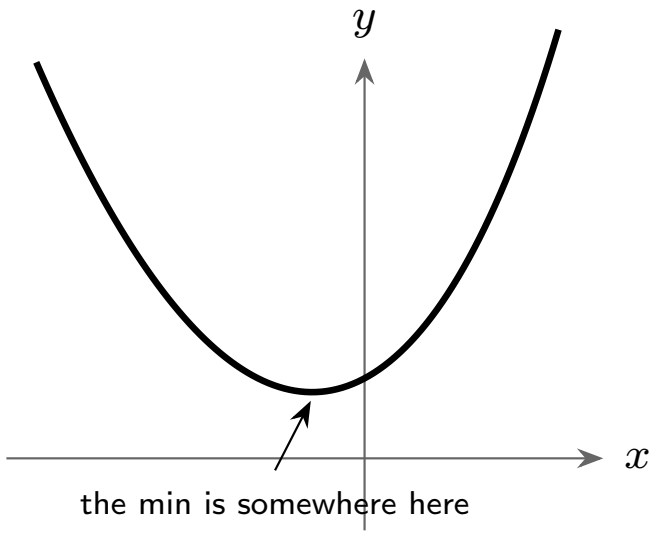
Gradient Descent

When You Cannot Solve $f'(x) = 0$

A harmless-looking function

$$f(x) = e^x + x^2 \quad \Rightarrow \quad f'(x) = e^x + 2x$$

- The minimum is where $e^x + 2x = 0$.
- No algebra isolates x in that equation.
- The derivative test **stalls at step 2**.



This is the usual case in machine learning

- The normal equations worked because a line with squared error has **linear** partials.
- Most models have thousands to billions of inputs, and no formula for $f' = 0$.
- We need a method that only **evaluates** f' at a point, and never **solves** for its zero.

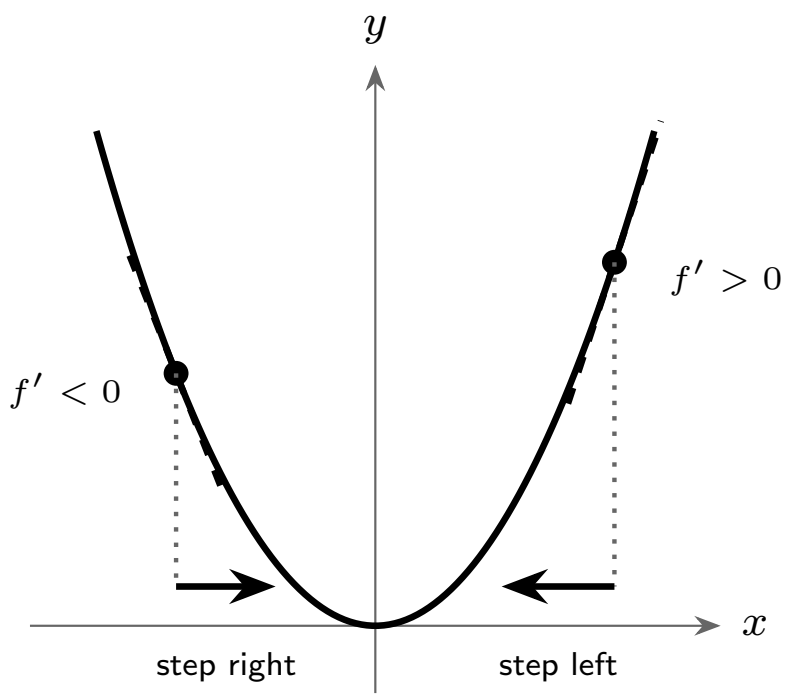
The Idea: Walk Downhill

Lost on a hillside in fog

1. Start somewhere: a **guess**.
2. Feel the slope under your feet.
3. Take a step **downhill**.
4. Repeat until the ground is flat.

Which way is downhill?

- $f'(x) > 0$: rising to the right, so move **left**.
- $f'(x) < 0$: falling to the right, so move **right**.
- Either way: move **against** the sign of $f'(x)$.



Dashed: tangent lines. Arrows: the step from each point, against the slope.

The Update Rule

Gradient descent, one step

$$x_{k+1} = x_k - \alpha\, f'(x_k)$$

Reading each piece

- x_k: where we stand now (step k). x_0 is the starting guess.
- $f'(x_k)$: the slope where we stand.
- α: the **learning rate**, a small positive number such as 0.1. It sets the step size. (The notebook writes η, eta.)
- The **minus sign**: step **against** the slope, which is downhill.

Two things to notice

- A steep slope gives a **big** step; a gentle slope gives a **small** one.
- At a minimum, $f'(x_k) = 0$, so $x_{k+1} = x_k$: the walk stops by itself.

Gradient Descent: the Algorithm

Algorithm: gradient descent

1. Choose a learning rate α, a maximum number of steps N, and a starting guess x_0.
2. For $k = 0, 1, \ldots, N-1$:
   - compute the slope $f'(x_k)$;
   - update $x_{k+1} = x_k - \alpha f'(x_k)$;
   - stop early if the step was tiny.
3. Return the last x.

When to stop

- After a fixed number of steps N.
- When the slope is tiny: $|f'(x_k)|$ below a small tolerance.
- When the steps are tiny: $|x_{k+1} - x_k|$ below a tolerance.

In practice: a tolerance, **plus** a cap N as a safety net.

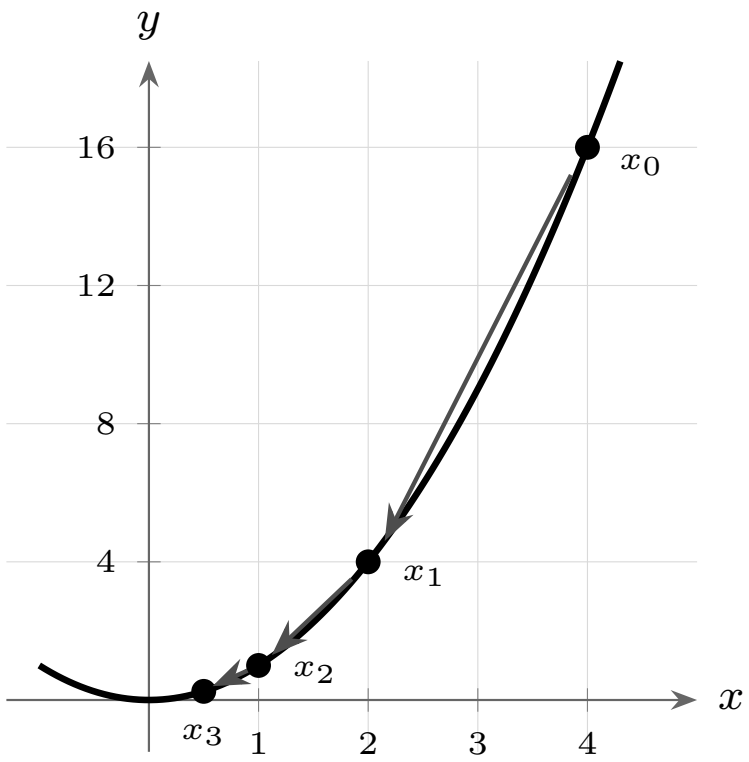
Worked: $f(x) = x^2$, $\alpha = 0.25$, $x_0 = 4$

Here $f'(x) = 2x$

k	x_k	$f'(x_k)$	$\alpha f'(x_k)$	x_{k+1}
0	4	8	2	2
1	2	4	1	1
2	1	2	0.5	0.5

What to notice

- Each step: $x_{k+1} = x_k - 0.25 \cdot 2x_k = \frac{1}{2}x_k$, so x **halves**.
- Steps shrink, $2, 1, 0.5$, as the slope flattens near the bottom.
- It heads for the true minimum $x = 0$ without ever solving $2x = 0$.



Notebook `U3-7_GradientDescent-1_Descent`, §1

Choosing the Learning Rate

For $f(x) = x^2$ the update is $x_{k+1} = x_k - \alpha \cdot 2x_k = (1 - 2\alpha)\, x_k$. Start at $x_0 = 4$:

Same function, same start, four learning rates

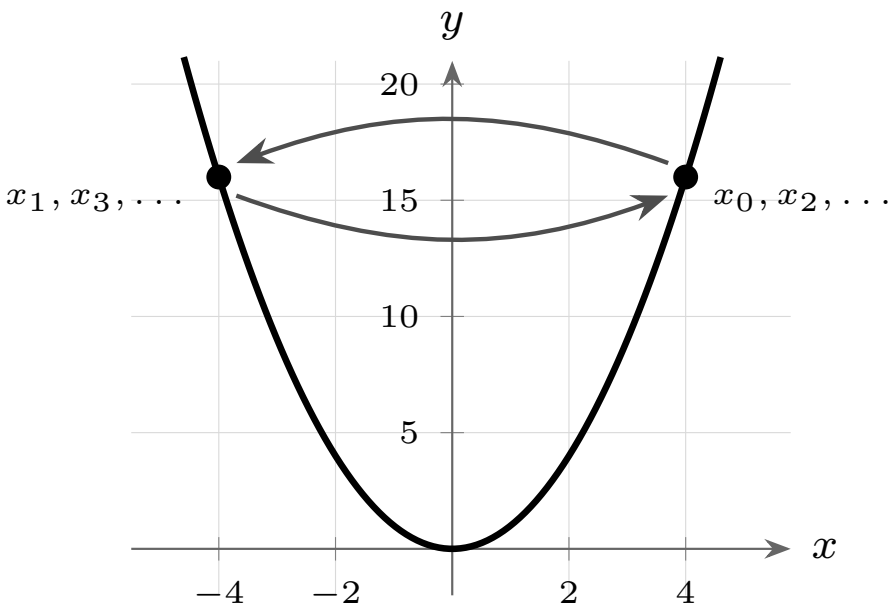
α	$1 - 2\alpha$	x_0	x_1	x_2	x_3	
0.05	0.9	4	3.6	3.24	2.916	**too small**: crawls
0.25	0.5	4	2	1	0.5	**good**: steady
1	-1	4	-4	4	-4	**too big**: bounces forever
1.25	-1.5	4	-6	9	-13.5	**too big**: blows up

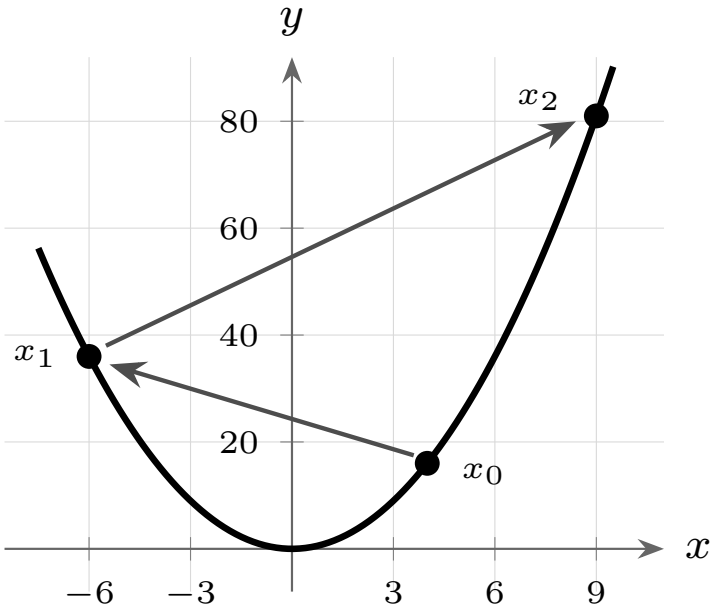
The tradeoff

- Too small: correct, but it may need thousands of steps.
- Too big: each step **overshoots** the bottom and lands on the far side, possibly higher than before.
- There is no universal right α: in practice you try a few and watch $f(x_k)$ fall.

Notebook `U3-7_GradientDescent-1_Descent`, §3

Too Big: Overshooting





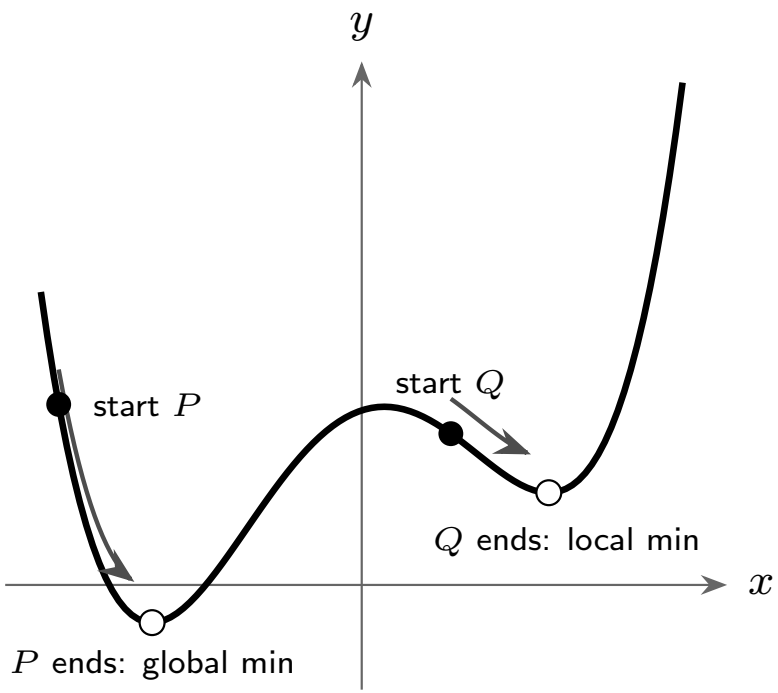
$\alpha = 1$: bounces forever

$4 \to -4 \to 4 \to \cdots$, never closer to 0.

$\alpha = 1.25$: blows up

$4 \to -6 \to 9$: each landing is **higher**.

The Local Minimum Trap



The start decides the end

- $f'(x) = 0$ at a local min too, so the walk stops there.
- Q starts above the shallow valley and never sees the deep one.

What people do about it

- **Restarts**: run from several starts, keep the lowest result.
- Smarter optimizers that build up speed to roll through small dips.

Notebook `U3-7_GradientDescent-1_Descent`, §4

Strengths and Weaknesses

Why it is used everywhere

- **Simple**: one line of arithmetic, repeated.
- Needs only to **evaluate** f' at a point, never to solve $f'(x) = 0$.
- Steps scale with the slope: big when far away, small near the bottom.
- Works the same with millions of inputs; this lecture takes it to two.

What can go wrong

- Gets **stuck** in local minima.
- A bad learning rate: too small crawls, too big bounces or blows up.
- Needs the derivative, as a formula or a numerical approximation.
- In very deep models the slopes can become huge or nearly zero, so the steps blow up or stall.

Gradient Descent in Python

Minimizing $f(x) = e^x + x^2$

```
import numpy as np

def df(x):                  # slope of f
    return np.exp(x) + 2*x

x = 1.0                     # starting guess x_0
alpha = 0.1                 # learning rate
for k in range(100):        # N = 100 steps
    x = x - alpha * df(x)

print(x)
# -0.3517337112491751
print(df(x))
# 5.60662627435704e-14
```

Reading the code

- The loop body **is** the update rule.
- We only ever **evaluate** `df(x)`.

Check the answer

`e-14` means $\times 10^{-14}$: the final slope is essentially 0, so the ground is flat. We found where $e^x + 2x = 0$ without solving it.

Notebook `U3-7_GradientDescent-1_Descent`, §2

Gradient Descent in Two Variables

Gradient Descent in Two Variables

From one variable to two

$$x_{k+1} = x_k - \alpha f'(x_k) \qquad \longrightarrow \qquad \begin{bmatrix} x_{k+1} \\ y_{k+1} \end{bmatrix} = \begin{bmatrix} x_k \\ y_k \end{bmatrix} - \alpha \nabla f(x_k, y_k)$$

Same idea: step **against** the slope. The **gradient** (Unit 3-1) points steepest uphill, so $-\nabla f$ is the way down.

The algorithm

1. Choose a learning rate α, a number of steps N, and a start (x_0, y_0).
2. Compute the gradient at the current point.
3. Step against it: subtract $\alpha \nabla f$ from the current point.
4. Repeat 2 and 3 until ∇f is nearly $\bar{0}$ or N steps are done.

Same cautions as in one variable: α too small crawls; too big overshoots or blows up. It can still stop in a local minimum.

Notebook `U3-7_GradientDescent-1_Descent`, §1 and §3 (one variable)

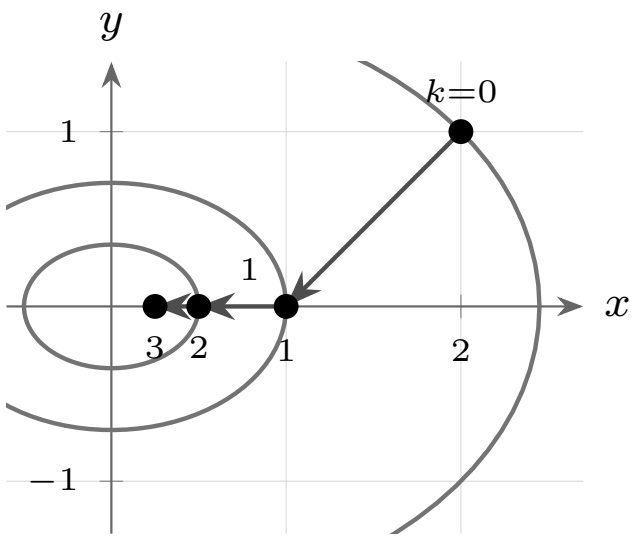
Worked: Descent on $x^2 + 2y^2$

Start $(2, 1)$, $\alpha = 0.25$

$\nabla f = \left[\begin{smallmatrix} 2x \\ 4y \end{smallmatrix}\right]$. Step 1:

$$\begin{bmatrix} 2 \\ 1 \end{bmatrix} - 0.25 \begin{bmatrix} 4 \\ 4 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

k	(x_k, y_k)	∇f	f
0	$(2, 1)$	$\left[\begin{smallmatrix} 4 \\ 4 \end{smallmatrix}\right]$	6
1	$(1, 0)$	$\left[\begin{smallmatrix} 2 \\ 0 \end{smallmatrix}\right]$	1
2	$(0.5, 0)$	$\left[\begin{smallmatrix} 1 \\ 0 \end{smallmatrix}\right]$	0.25
3	$(0.25, 0)$	$\left[\begin{smallmatrix} 0.5 \\ 0 \end{smallmatrix}\right]$	0.0625



Labels are step numbers k.

What happened

f drops at every step. y reaches 0 in one step; after that x halves each step, heading for the minimum at $(0, 0)$.

Solving Linear Regression: Gradient Descent

Gradient Descent on the Loss

The update

$$\begin{bmatrix} m \\ b \end{bmatrix} \leftarrow \begin{bmatrix} m \\ b \end{bmatrix} - \alpha \nabla L, \qquad \nabla L = \begin{bmatrix} \partial L/\partial m \\ \partial L/\partial b \end{bmatrix} = \begin{bmatrix} \sum x_i(\hat{y}_i - y_i) \\ \sum(\hat{y}_i - y_i) \end{bmatrix}$$

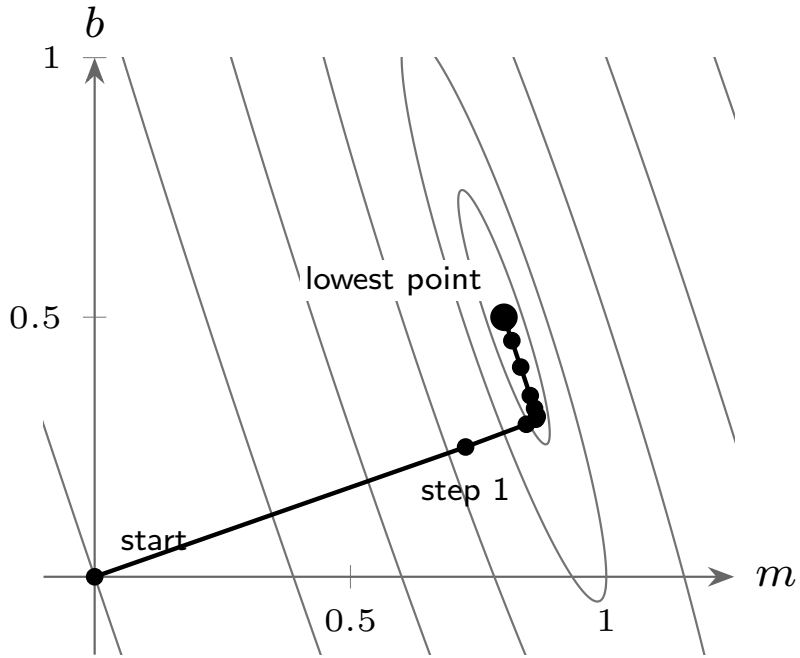
One step from $(m, b) = (0, 0)$ with $\alpha = 0.025$

- The line $\hat{y} = 0$ predicts 0, so $\hat{y}_i - y_i$ is $-1, -3, -2, -4$.
- $\dfrac{\partial L}{\partial m} = 1(-1) + 2(-3) + 3(-2) + 4(-4) = -29$
- $\dfrac{\partial L}{\partial b} = -1 - 3 - 2 - 4 = -10$
- New point: $m = 0 - 0.025(-29) = 0.725, \quad b = 0 - 0.025(-10) = 0.25.$

Meaning

Both partials are negative, so the step raises both. The loss falls from 15 to about 1.30.

The Path to the Bottom



Contours of $L(m, b)$; dots are steps.

$\alpha = 0.025$, from $(0, 0)$

k	(m_k, b_k)	L
0	$(0, 0)$	15
1	$(0.725, 0.250)$	1.2969
2	$(0.844, 0.294)$	0.9236
10	$(0.860, 0.324)$	0.9103
100	$(0.815, 0.455)$	0.9007
1000	$(0.800, 0.500)$	0.9000

What happened

- Two steps drop straight down the steep wall of the valley. Then a slow crawl along its narrow floor.
- It ends at $(0.8, 0.5)$: the **same line** as the normal equations.

Notebook `U3-7_Regression-3_ThreeWays`, §2

Normal Equations or Gradient Descent?

For linear regression, solve; do not descend

- The normal equations give the exact (m, b) in **one** linear solve.
- Gradient descent took about 1000 steps to creep to the same point, and needed a well-chosen α.
- So for a line with squared error, gradient descent is the **slow way**. Use the formula.

Why the formula exists, and when it does not

- A line with squared error has partials that are **linear** in m and b, so $\nabla L = \bar{0}$ is a linear system (Unit 2-1).
- Change the function and that breaks: the partials become **nonlinear**, and no algebra solves $\nabla f = \bar{0}$.
- Gradient descent only needs to **evaluate** ∇f where you stand. That is why logistic regression and neural networks, including the language models behind chatbots, are trained with it.

Gradient Descent on a Nonlinear Function

$f(x, y) = x^4 + 2y^2 + e^{x-y}$

$$\partial f/\partial x = 4x^3 + e^{x-y} = 0$$
$$\partial f/\partial y = 4y - e^{x-y} = 0$$

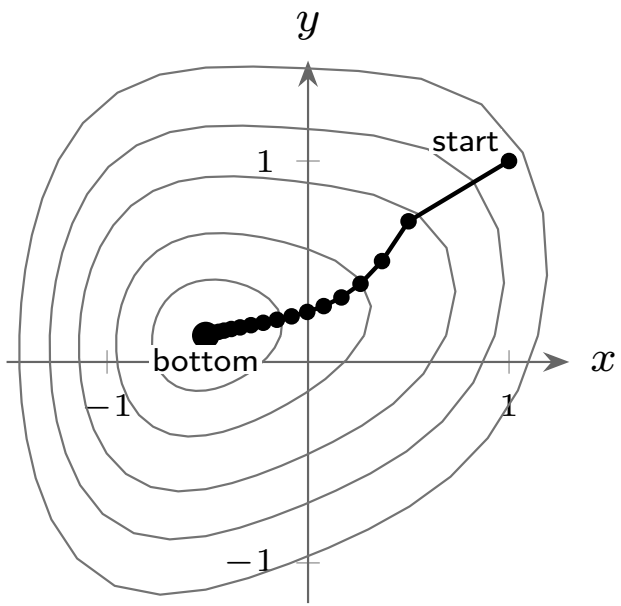
A cube tangled with an exponential: **not** a linear system, and no algebra isolates x or y.

But we can still walk downhill

From $(1, 1)$ with $\alpha = 0.1$:
$\left[\begin{smallmatrix}1\\1\end{smallmatrix}\right] - 0.1\left[\begin{smallmatrix}5\\3\end{smallmatrix}\right] = \left[\begin{smallmatrix}0.5\\0.7\end{smallmatrix}\right]$.

k	(x_k, y_k)	f
0	$(1, 1)$	4.000
1	$(0.5, 0.7)$	1.861
10	$(-0.284, 0.183)$	0.700
50	$(-0.509, 0.132)$	0.629

At step 50, $\nabla f \approx \bar{0}$: the bottom, found without solving anything.



Contours of f at $0.8, 1.2, 2, 3, 4.5$; dots are steps.

Linear Regression in Python

1. Normal equations (NumPy)

```
import numpy as np
x = np.array([1, 2, 3, 4])
y = np.array([1, 3, 2, 4])
A = np.array([[np.sum(x*x), np.sum(x)],
              [np.sum(x),   len(x)]])
r = np.array([np.sum(x*y), np.sum(y)])
np.linalg.solve(A, r)
# array([0.8, 0.5])
```

2. Minimize the loss (SciPy)

```
from scipy.optimize import minimize
def L(w):                  # w = [m, b]
    m, b = w
    return 0.5 * np.sum((m*x + b - y)**2)
res = minimize(L, [0, 0])  # start: m = b = 0
res.x.round(3)
# array([0.8, 0.5])
```

3. The library (scikit-learn)

```
from sklearn.linear_model \
    import LinearRegression
X = x.reshape(-1, 1)
model = LinearRegression()
model.fit(X, y)
model.coef_, model.intercept_
# 0.8 and 0.5, up to rounding
```

Notes

- `np.sum` is the $\sum$.
- `minimize` walks downhill on L from a starting guess.
- `reshape(-1, 1)`: one column per feature.
- All three give the same line.

Notebook `U3-7_Regression-3`, §2 to §4

Review: Optimization and Linear Regression

What we covered: finding extrema with calculus

1. Optimization finds the min or max of an **objective function** f over a **search space** D. The **minimizer** x^* says where; the **minimum value** $f(x^*)$ says how good. A max of f is a min of $-f$.
2. A **global** minimum is lowest over all of D; a **local** one is lowest only nearby. Algorithms can mistake one for the other, and with many inputs we cannot plot to check.
3. **Derivative test**: at a smooth extremum inside D, $f'(x) = 0$. Solve for the critical points, add any endpoints, and compare values.
4. $f'(x) = 0$ gives candidates, not answers (x^3 at 0). With a restricted D, the best value can sit at an endpoint where $f' \neq 0$.
5. **Second derivative test**: at a critical point, $f'' > 0$ means a local min, $f'' < 0$ a local max, and $f'' = 0$ says nothing. If f is convex everywhere, a critical point is the global min.

Review: Optimization and Linear Regression

What we covered: the best line

6. The model $\hat{y} = mx + b$; residual $e_i = \hat{y}_i - y_i$; loss $L(m, b) = \frac{1}{2}\sum(\hat{y}_i - y_i)^2 = \frac{1}{2}\sum e_i^2$. Squaring stops signs cancelling, makes big misses cost more, and, unlike $|e|$, leaves a slope to set to 0 at the bottom.
7. The data are fixed, so L is a function of m and b: a **bowl** whose one lowest point is the best line.
8. **Normal equations**: setting $\frac{\partial L}{\partial m} = \frac{\partial L}{\partial b} = 0$ gives a 2×2 linear system. In matrix form, $\left[\begin{smallmatrix} m \\ b \end{smallmatrix}\right] = (X^\top X)^{-1} X^\top \bar{y}$.

Review: Optimization and Linear Regression

What we covered: gradient descent

9. When $f'(x) = 0$ cannot be solved, **gradient descent** searches instead: $x_{k+1} = x_k - \alpha f'(x_k)$. Step against the slope, and repeat. It only ever evaluates f'.
10. Steps shrink on their own as the slope flattens; at a minimum the walk stops.
11. The **learning rate** α: too small crawls, too big overshoots, bounces, or blows up.
12. Stop after N steps, or when the slope or the step falls below a tolerance.
13. Gradient descent finds **a** local minimum, and which one depends on the start. Restarts help.
14. **In the plane**: $\begin{bmatrix} x_{k+1} \\ y_{k+1} \end{bmatrix} = \begin{bmatrix} x_k \\ y_k \end{bmatrix} - \alpha \nabla f(x_k, y_k)$, stepping against the gradient.

Review: Optimization and Linear Regression

What we covered: solving linear regression with gradient descent

15. **Gradient descent** on L reaches the same line. It still works when no formula exists, which is why it trains every neural network.

Notebooks: `U3-5_Optimize-1`; `U3-7_Regression-1` to `-6`; `U3-7_GradientDescent-1`.
Previous: Unit 3-1, Limits and Derivatives. **Next:** Unit 3-3, **Integrals**.